

AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH

Faculty of Science and Technology (FST)

Department of Computer Science (CS)

Undergraduate Program

LAB MANUAL

Programming in Python

Course code and title:	CSC 4162: Programming in Python
Credit:	3 credit hours
Weekly contact hours:	2 hours of theory and 2 hours 20 minutes of laboratory

Contents

Lab 02: Python Installation and First Programs **1**

Lab 03: Variables, Data Types, Operators, and Branching **2**

Lab 04: Loops **6**

Lab 05: Functions and Lists **9**

Lab 06: Classes, Objects, and Inheritance **11**

Lab 07: Exception Handling and File Operations **13**

Lab 02

Python Installation and First Programs

Topics

- Installing Python.
- Start programming using the Python environment.

Background

There are different ways to write code in Python. In this course, we will see several options such as IDLE, Spyder, PyCharm, Jupyter Notebook, and Google Colab. However, we will mostly use IDLE and Google Colab to discuss our example codes.

Declaring Variables

Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"

print(x)
print(y)
```

Input and Output Values

Python uses the `input()` method to ask for a user input. The `input()` function returns the input value as a string.

The Python `print()` function is often used to output variables.

```
username = input("Enter username:")

print("Username is:", username)
print("Username is: " + username)
print(type(username))
```

List of Problems

Problem 1. Demonstrate the installation steps of Python. The latest Python can be downloaded from: https://www.python.org/downloads/.

Problem 2. Introduction with IDLE and different coding environments such as PyCharm, Spyder, Jupyter Notebook, and Google Colab.

Problem 3. Explore IDLE and Google Colab using the previous basic programming knowledge.

Lab 03

Variables, Data Types, Operators, and Branching Statements

Topics

- Variables, Data types, and Operators.
- Branching statements.

Background

Variable Names

Rules for Python variables:

- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alphanumeric characters and underscores (A–Z, a–z, 0–9, and _).
- Variable names are case-sensitive (`age`, `Age`, and `AGE` are three different variables).
- A variable name cannot be any of the Python keywords.

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

Global Variables

Variables that are created outside of a function (as in all of the examples so far) are known as global variables. Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"

def myfunc():
    print("Python is " + x)

myfunc()
```

Data Types

Python has the following data types built-in by default, in these categories:

Category	Data types
Text Type	`str`
Numeric Types	`int`, `float`, `complex`
Sequence Types	`list`, `tuple`, `range`
Mapping Type	`dict`
Set Types	`set`, `frozenset`
Boolean Type	`bool`
Binary Types	`bytes`, `bytearray`, `memoryview`
None Type	`NoneType`

Example	Example (specifying the data type)	Data type
`x = "Hello World"`	`x = str("Hello World")`	`str`
`x = 20`	`x = int(20)`	`int`
`x = 20.5`	`x = float(20.5)`	`float`
`x = 1j`	`x = complex(1j)`	`complex`
`x = ["apple", "banana", "cherry"]`	`x = list(("apple", "banana", "cherry"))`	`list`
`x = ("apple", "banana", "cherry")`	`x = tuple(("apple", "banana", "cherry"))`	`tuple`
`x = "name": "John", "age": 36`	`x = dict(name="John", age=36)`	`dict`
`x = "apple", "banana", "cherry"`	`x = set(("apple", "banana", "cherry"))`	`set`
`x = True`	`x = bool(5)`	`bool`

Operators

Arithmetic Operators

Operator	Name	Example
`+`	Addition	`x + y`
`-`	Subtraction	`x - y`
`*`	Multiplication	`x * y`
`/`	Division	`x / y`
`%`	Modulus	`x % y`
`**`	Exponentiation	`x ** y`
`//`	Floor division	`x // y`

Comparison Operators

Operator	Name	Example
`==`	Equal	`x == y`
`!=`	Not equal	`x != y`
`>`	Greater than	`x > y`
`<`	Less than	`x < y`
`>=`	Greater than or equal to	`x >= y`
`<=`	Less than or equal to	`x <= y`

Logical Operators

Operator	Description	Example
`and`	Returns True if both statements are true	`x < 5 and x < 10`
`or`	Returns True if one of the statements is true	`x < 5 or x < 4`
`not`	Reverse the result, returns False if the result is true	`not(x < 5 and x < 10)`

Identity Operators

Operator	Description	Example
`is`	Returns True if both variables are the same object	`x is y`
`is not`	Returns True if both variables are not the same object	`x is not y`

Membership Operators

Operator	Description	Example
`in`	Returns True if a sequence with the specified value is present in the object	`x in y`
`not in`	Returns True if a sequence with the specified value is not present in the object	`x not in y`

Operator Precedence

Operator	Description
`()`	Parentheses
`**`	Exponentiation
`+x -x ~x`	Unary plus, unary minus, and bitwise NOT
`* / // %`	Multiplication, division, floor division, and modulus
`+ -`	Addition and subtraction
`« »`	Bitwise left and right shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`== != > >= < <= is is not in not in`	Comparisons, identity, and membership operators
`not`	Logical NOT
`and`	AND
`or`	OR

Branching Statements

In Python, `if-else` or `if-elif-else` statements are mostly used for branching.

If Statement

```
a = 33
b = 200

if b > a:
    print("b is greater than a")
```

If–else

```
a = 200
b = 33

if b > a:
    print("b is greater than a")
else:
    print("a is greater than b")
```

If–elif

```
a = 33
b = 33

if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

If–elif–else

```
a = 200
b = 33

if b > a:
    print("b is greater than a")
```

```
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

Match Statement

```
day = 4

match day:
    case 6:
        print("Today is Saturday")
    case 7:
        print("Today is Sunday")
    case _:
        print("This is Weekend")
```

List of Problems

Problem 1. Write a Python program to swap the values of two variables.

Problem 2. Write a program to compute the distance between two points, taking input from the user. Distance formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Problem 3. Write a program to check whether the given number is an even or odd number.

Problem 4. Take a character as input and determine whether it is a vowel. Print an appropriate message (vowel/not vowel).

Problem 5. Write a program to convert a student's marks of a course into a grade. Use the grading criteria of our university.

Lab 04

Loops

Topics

- Loops.

Background

Python has two primitive loop commands:

- `while` loops.
- `for` loops.

`for` Loops

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
    print(x)
```

```
for x in "banana":
    print(x)
```

```
fruits = ["apple", "banana", "cherry"]

for ind, val in enumerate(fruits):
    print(ind, val)
```

`for` Loops: `break` and `continue`

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
    if x == "banana":
        break
    print(x)
```

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
    if x == "banana":
        continue
    print(x)
```

`for` Loops: `range()` Function

```
for x in range(2, 10, 3):
    print(x)
```

`for` Loops: `else`

The `else` keyword in a `for` loop specifies a block of code to be executed when the loop is finished.

The `else` block will **not** be executed if the loop is stopped by a `break` statement.

```
for x in range(6):
    print(x)
else:
    print("Finally finished!")
```

Nested `for` Loops

```
adj = ["big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

`while` Loops

```
i = 1

while i < 6:
    print(i)
    i += 1
```

```
i = 1

while i < 6:
    print(i)

    if i == 3:
        break

    i += 1
```

```
i = 0

while i < 6:
    i += 1

    if i == 3:
        continue

    print(i)
```

```
i = 1

while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

List of Problems

Problem 1. Write a program using a `for` loop that loops over a sequence.

Problem 2. Write a program using a `while` loop that asks the user for a number and prints a countdown from that number to zero.

Problem 3. Write a Python program to find the exponentiation of a number.

Problem 4. Find the sum of all the primes below 1000.

Problem 5. Print Fibonacci numbers below a given user input N.

Lab 05

Functions and Lists

Topics

- Defining and calling functions; parameters and return values.
- Lists, indexing, iteration, and list operations.
- Combining loops, conditional statements, and functions.

Background

Functions divide a program into smaller, reusable blocks of code. In Python, functions are defined using the `def` keyword. Parameters allow functions to receive input, while the `return` statement sends a result back to the caller. Lists store multiple items in a single variable. They support indexing, iteration, and operations that allow programs to process and analyze collections of data.

Exercise 1: Campus Resource Tracker

Problem 1. A university wants to track the availability of resources such as projectors, laptops, and laboratory equipment. Develop a program that maintains resource information and helps users identify available resources.

Each resource should contain a unique ID, a name, a category, and the available quantity. You may represent each resource using a list or use a list of dictionaries if dictionaries have already been covered.

Sample data

```
resources = [
    [101, "Laptop", "Electronics", 8],
    [102, "Projector", "Electronics", 0],
    [103, "Whiteboard", "Classroom", 5],
    [104, "Microphone", "Audio", 3],
    [105, "Camera", "Electronics", 2]
]
```

Tasks

1. Write a function `display_resources(resources)` to display all resources with their details.
2. Write a function `search_by_category(resources, category)` to display all resources belonging to a given category.
3. Write a function `find_unavailable(resources)` to identify resources whose available quantity is zero.

Testing and Debugging

Test your program with an unknown resource ID, a negative quantity, a resource with zero units, and an empty resource list. Explain how your program handles each case.

Exercise 2: Student Marks Investigator

Problem 2. An instructor wants to analyze students' assessment marks to identify performance patterns. Develop a program that processes student records and reports useful information about their results.

Each student record contains a student ID, a name, and marks from three assessments. Marks must be between 0 and 100.

Sample data

```
students = [
    [201, "Amina", [78, 85, 90]],
    [202, "Rahim", [65, 70, 68]],
    [203, "Nadia", [92, 88, 95]],
    [204, "Hasan", [55, 60, 65]],
    [205, "Tania", [80, 80, 80]]
]
```

Tasks

1. Write a function `calculate_average(marks)` that returns the average of a student's marks.
2. Write a function `classify_performance(average)` that returns a performance category using these rules:
   - Average ≥ 80: Excellent
   - Average ≥ 70: Good
   - Average ≥ 60: Satisfactory
   - Average < 60: Needs Improvement
3. Write a function `display_report(students)` that displays each student's ID, name, average, and performance category.

Testing and Debugging

Test your program with a student who has equal marks in all assessments, students with tied averages, an invalid mark, and a student record with no marks. Decide how invalid or missing data should be handled and implement the appropriate checks.

Lab 06

Classes, Objects, and Inheritance

Topics

- Classes, objects, attributes, and methods.
- Constructors and inheritance.
- Method overriding and `super()`.

Background

A class is a blueprint for creating objects. Objects contain attributes (data) and methods (functions). The `__init__()` method initializes an object, while `self` refers to the current instance. Inheritance allows a child class to reuse and override methods of a parent class.

Example 1: Classes and Objects

```python
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id

    def show_info(self):
        print(self.name, self.student_id)

s1 = Student("John", "22-12345-1")
s1.show_info()
```

Example 2: Inheritance

```python
class Vehicle:
    def __init__(self, name, seating_capacity):
        self.name = name
        self.seating_capacity = seating_capacity

    def fare(self):
        return self.seating_capacity * 100


class Bus(Vehicle):
    def fare(self):
        amount = super().fare()
        return amount * 1.10

b1 = Bus("AIUB Bus", 40)
print(b1.fare())  # Output: 4400.0
```

Example 3: Polymorphism

Polymorphism allows different classes to implement the same method in different ways. For example, different animals can make different sounds using the same method name.

```
class Dog:
    def sound(self):
        print("Bark")


class Cat:
    def sound(self):
        print("Meow")


animals = [Dog(), Cat()]

for animal in animals:
    animal.sound()
```

Output

```
Bark
Meow
```

List of Problems

Problem 1. Create a `BankAccount` class with attributes `account_number`, `balance`, `date_of_opening`, and `customer_name`. Implement methods to deposit money, withdraw money, and check the balance. Reject invalid amounts and withdrawals exceeding the available balance.

Problem 2. Create a `Bus` class that inherits from `Vehicle`. The base fare is `seating_capacity * 100`. Add a 10% maintenance charge to calculate the final bus fare.

Problem 3. Create a parent class `Shape` with a method `area()`. Create two child classes, `Rectangle` and `Circle`, that implement `area()` differently. Calculate and display the area of each shape.

Lab 07

Exception Handling and File Operations

Topics

- Exception handling: `try`, `except`, `else`, and `finally`.
- File operations: reading, writing, appending, and creating.

Background

Exceptions are errors that occur during program execution. Python uses `try-except` to handle exceptions. The `else` block runs when no exception occurs, while `finally` always executes.

Example 1: Exception Handling

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Program ended.")
```

Example 2: File Operations

Common file modes are `r` (read), `w` (write), `a` (append), and `x` (create a new file). Using `with` automatically closes the file.

```
with open("sample.txt", "w", encoding="utf-8") as file:
    file.write("Hello Python!")

with open("sample.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

List of Problems

Problem 1. Write a program that divides a number by zero and handles the resulting `ZeroDivisionError`.

Problem 2. Ask the user to enter two numbers and perform an arithmetic operation. Handle invalid numerical input using `ValueError`. Handle division by zero if applicable.

Problem 3. Access a list element using a user-provided index. Handle an out-of-range index using `IndexError` and invalid integer conversion using `ValueError`.

Problem 4. Create a file in a specified directory and practise reading, writing, appending, and creating files. Use `try-except` to handle errors such as missing files, missing directories, and existing files.